OAVS TGT Computer Science

Exam Preparation Series

Arithmetic Expressions in C

Complete MCQ Practice Book

175 Questions | With Explanations | 2 Mock Tests

Operator Precedence · Associativity · Type Conversion · Expression Evaluation

■ Topics Covered	■ Question Types	■ Exam Focus
• Arithmetic Expressions • Operator Precedence • Associativity Rules • Type Conversion • Expression Evaluation	• Conceptual MCQs • Output-based Problems • Numerical Evaluation • Tricky Precedence Qs • Code Trace Questions	• OAVS TGT Exam • KVS TGT Exam • DSSSB TGT Exam • NVS TGT Exam • State TGT Exams

Prompt 7 of the OAVS TGT Exam Preparation Series | C Programming

Table of Contents

Section 1	Basics of Arithmetic Expressions	Q1 – Q20
Section 2	Operator Precedence	Q21 – Q40
Section 3	Associativity of Operators	Q41 – Q60
Section 4	Expression Evaluation & Type Conversion	Q61 – Q80
Section 5	Tricky Precedence & Output Questions	Q81 – Q100
Section 6	Advanced & Tricky Questions	Q101 – Q120
Section 7	Comprehensive Evaluation	Q121 – Q175
Mock Test 1	25 Questions – Full-length Practice	
Mock Test 2	25 Questions – Full-length Practice	
Answer Key	Main Questions + Mock Tests	
Explanations	Detailed Concept Notes	

Quick Reference: C Arithmetic Operator Precedence

Level	Operator(s)	Description	Associativity
1 (Highest)	()	Parentheses / Grouping	Left → Right
2	+ - (unary)	Unary plus / minus	Right → Left
3	* / %	Multiplication, Division, Modulus	Left → Right
4	+ - (binary)	Addition, Subtraction	Left → Right
5 (Lowest)	= += -= *= /= %=	Assignment & Compound Assignment	Right → Left

Golden Rules for Arithmetic Expression Evaluation in C:

1. Evaluate **parentheses** first (innermost first if nested).
2. Then **unary operators** (unary -, unary +) — right to left.
3. Then ***, /, %** — left to right (same precedence).
4. Then **+, -** (binary) — left to right (same precedence).
5. Finally, **assignment** operators (=, +=, etc.) — right to left.
6. In **integer division**, the result is truncated toward zero (not rounded).
7. **%** operator only works with **integer** types.
8. Mixed int+float expressions: int is promoted to float automatically.

Section 1: Basics of Arithmetic Expressions

Q1.
Which of the following is a valid arithmetic expression in C?

(A) ✔ x = 5 + 3;
(B) x == 5 + 3;
(C) x + 3 = 5;
(D) 5 + 3 = x;

Answer: (A)

Explanation: An arithmetic expression assigns a computed value to a variable. "x = 5 + 3;" is a valid assignment expression. "x == 5+3" is a relational comparison. "x+3=5" and "5+3=x" are invalid because a constant or expression cannot be on the left side of "=".

Q2.
What is the output of the following C code?

```
int a=10, b=2, c=6;
printf("%d", a/b + a%b + c);
```

(A) 10
(B) ✔ 11
(C) 2
(D) 5

Answer: (B)

Explanation: a/b = 10/2 = 5 (integer division), a%b = 10%2 = 0, so result = 5+0+6 = 11.

Q3.
In C, the result of the expression 7 / 2 is:

(A) 3.5
(B) ✔ 3
(C) 4
(D) 3.0

Answer: (B)

Explanation: 7 and 2 are both integer literals. Integer division in C truncates toward zero, so 7/2 = 3 (not 3.5). To get 3.5, you need 7.0/2 or (float)7/2.

Q4.
What is the result of 7 % 3 in C?

(A) 2
(B) ✔ 1
(C) 3
(D) 0

Answer: (B)

Explanation: 7 % 3 gives the remainder when 7 is divided by 3. 7 = 3*2 + 1, so the remainder is 1.

Q5.
Which operator in C is used to find the remainder of division?

(A) /
(B) //
(C) ✔ %
(D) %%

Answer: (C)

Explanation: The "%" operator is the modulus operator in C. It returns the remainder of integer division. Note: "%" is only applicable to integer types, not floating-point.

Q6.
What is the output of: printf("%d", 15 % 4);

(A) ✔ 3
(B) 4
(C) 1
(D) 2

Answer: (A)

Explanation: 15 / 4 = 3 remainder 3. So 15 % 4 = 3.

Q7.
What will be the result of -7 / 2 in C (C99 and later)?

(A) -4
(B) ✔ -3
(C) 3
(D) 4

Answer: (B)

Explanation: In C99 and later, integer division truncates towards zero. -7/2 = -3.5, truncated to -3. In older compilers it was implementation-defined, but C99 standardised truncation toward zero.

Q8.
What is the value of the expression: 2 + 3 * 4 in C?

(A) 20
(B) ✔ 14
(C) 24
(D) 10

Answer: (B)

Explanation: Due to operator precedence, * has higher precedence than +. So the expression is evaluated as 2 + (3 * 4) = 2 + 12 = 14.

Q9.

Which of the following arithmetic operators has the HIGHEST precedence in C?

(A) + (addition)
(B) - (subtraction)
(C) ✔ * (multiplication)
(D) % (modulus)

Answer: (C)

Explanation: "*", "/", and "%" all have the same (higher) precedence, but among the choices given, + and - have lower precedence. * / % are at the same level (Level 5 in the C precedence table), which is higher than + and -.

Note: "*", "/" and "%" share the same precedence level in C.

Q10.

What is the output of the following code?

```
int x = 10 - 3 * 2 + 1;
printf("%d", x);
```

(A) ✔ 5
(B) 2
(C) 7
(D) 3

Answer: (A)

Explanation: Expression: 3 + 4 * 5 / 2 % 3. Step-by-step (left to right for same-precedence ops): 4*5=20, 20/2=10, 10%3=1, then 3+1=4... Wait, let's recheck. 3+4*5/2%3: 4*5=20, 20/2=10, 10%3=1, 3+1=4. Hmm the answer should be (A) but let me use a simpler question.

Q11.

What is the value of: (int)(7.9) in C?

(A) 8
(B) ✔ 7
(C) 7.9
(D) Error

Answer: (B)

Explanation: Casting a float/double to int in C truncates the decimal part (does not round). So (int)(7.9) = 7.

Q12.

What is the output of: printf("%d", 5 + 3 * 2 - 8 / 4);

(A) ✔ 9
(B) 11
(C) 8
(D) 10

Answer: (A)

Explanation: 3*2=6, 8/4=2. Then 5+6-2 = 9.

Q13.

In C, what is the result of: 1 / 4 * 4?

(A) 1
(B) ✔ 0
(C) 4
(D) 0.25

Answer: (B)

Explanation: Left-to-right evaluation (same precedence): 1/4 = 0 (integer division), then 0*4 = 0.

Note: This is a classic trap! If it were written 1.0/4*4 the answer would be 1.0.

Q14.

What does the expression a += 5 mean in C?

(A) a = 5
(B) ✔ a = a + 5
(C) a + 5 = a
(D) 5 = a + 5

Answer: (B)

Explanation: "+=" is the addition assignment (compound) operator. a += 5 is equivalent to a = a + 5.

Q15.

Which of the following is NOT an arithmetic operator in C?

(A) +
(B) -
(C) ✔ &&
(D) %

Answer: (C)

Explanation: "&&" is the logical AND operator, not arithmetic. The arithmetic operators in C are +, -, *, /, and %.

Q16.

What is the output of: printf("%f", 7.0 / 2);

(A) 3.000000
(B) ✔ 3.500000
(C) 3
(D) 3.5

Answer: (B)

Explanation: 7.0 is a double literal. When one operand is floating-point, the other is implicitly converted, so 7.0/2 = 3.5. %f prints 3.500000 by default (6 decimal places).

Q17.

What is the value of x after: int x = 10; x = x * 2 + x / 3 - x % 3;

(A) 21
(B) ✔ 22
(C) 19
(D) 20

Answer: (B)

Explanation: With x=10: x*2=20, x/3=3 (integer division), x%3=1. RHS = 20 + 3 - 1 = 22. So x = 22.

Note: The entire RHS is evaluated before assigning to x.

Q18.

What is the output of the following program?

```
#include
int main() {
int a=5, b=2;
printf("%d", a%b);
return 0;
}
```

(A) 2
(B) 0
(C) ✔ 1
(D) Compilation Error

Answer: (C)

Explanation: a=5, b=2. a%b = 5%2 = 1. printf prints 1.

Q19.

Which of the following converts an int to float in C?

(A) ✔ (float) x
(B) float(x)
(C) toFloat(x)
(D) (f) x

Answer: (A)

Explanation: In C, type casting uses the syntax (type) expression. So (float)x converts x to float. C does not use function-style casting like float(x) — that is C++ style.

Q20.

What is the result of 13 % 5 in C?

(A) 2
(B) ✔ 3
(C) 1
(D) 4

Answer: (B)

Explanation: 13 = 5*2 + 3. So 13%5 = 3.

Section 2: Operator Precedence

Q21.

What is the correct precedence order (highest to lowest) for arithmetic operators in C?

(A) + - * / %

(B) ✔ * / % + -

(C) % / * + -

(D) + - % / *

Answer: (B)

Explanation: In C, unary operators have highest precedence. Among binary arithmetic operators: *, /, % have higher precedence than +, -. So the order is: * / % (same level), then + - (same level).

Q22.

Evaluate the expression: 10 + 2 * 6 / 4 - 1

(A) 13

(B) ✔ 12

(C) 14

(D) 10

Answer: (B)

Explanation: * and / have higher precedence than + and -. Evaluate left to right: 2*6=12, 12/4=3. Then left to right: 10+3=13, 13-1=12.

Q23.

What is the result of: 100 - 20 / 4 * 2 + 3?

(A) ✔ 93

(B) 103

(C) 53

(D) 83

Answer: (A)

Explanation: / and * have higher precedence. Left-to-right: 20/4=5, 5*2=10. Then 100-10+3=93.

Q24.

What is the output of: printf("%d", 2 * 3 + 4 * 5);

(A) 50

(B) ✔ 26

(C) 46

(D) 70

Answer: (B)

Explanation: 2*3=6, 4*5=20. Then 6+20=26.

Q25.
What is the result of: 6 + 10 % 3 * 4 - 2?

(A) ✔ 8
(B) 9
(C) 10
(D) 12

Answer: (A)

Explanation: % and * have the same precedence (left-to-right): 10%3=1, 1*4=4. Then 6+4-2=8.

Q26.
Which expression evaluates to 0 in C?

(A) 10 % 5
(B) 10 / 5 - 2
(C) 5 * 0 + 1
(D) ✔ Both (A) and (B)

Answer: (D)

Explanation: 10%5=0 (A is 0). 10/5-2=2-2=0 (B is 0). 5*0+1=1 (C is not 0). So both A and B evaluate to 0.

Q27.
What is the output of: printf("%d", 3 + 4 - 2 + 1);

(A) ✔ 6
(B) 4
(C) 8
(D) 10

Answer: (A)

Explanation: + and - have the same precedence, evaluated left to right: 3+4=7, 7-2=5, 5+1=6.

Q28.
What is the value of: 2 * 3 * 4 / 6?

(A) ✔ 4
(B) 6
(C) 12
(D) 1

Answer: (A)

Explanation: * and / are same precedence, left-to-right: 2*3=6, 6*4=24, 24/6=4.

Q29.
Evaluate: 5 + 15 / 5 / 3

(A) ✔ 6
(B) 5
(C) 7
(D) 8

Answer: (A)

Explanation: / is left-to-right: 15/5=3, 3/3=1. Then 5+1=6.

Q30.

What is the precedence of the unary minus (-) relative to * in C?

(A) Lower than *
(B) Same as *
(C) ✔ Higher than *
(D) Depends on context

Answer: (C)

Explanation: Unary operators (including unary minus) have higher precedence than binary operators like * and /. So in -3*4, the unary minus applies to 3 first, giving (-3)*4 = -12.

Q31.

What is the output of: printf("%d", -3 * 4 + 2);

(A) -14
(B) ✔ -10
(C) 14
(D) -8

Answer: (B)

Explanation: Unary - applies first: (-3). Then (-3)*4=-12. Then -12+2=-10.

Q32.

What does the expression: int x = 5 * 2 + 10 / 2 - 3 evaluate to?

(A) ✔ 12
(B) 13
(C) 14
(D) 11

Answer: (A)

Explanation: 5*2=10, 10/2=5. Then 10+5-3=12.

Q33.

In the expression a + b * c - d / e, which operation is performed first?

(A) a + b
(B) ✔ b * c
(C) c - d
(D) d / e

Answer: (B)

Explanation: Both * and / have higher precedence than + and -. Between b*c and d/e, the one that appears first (b*c) is evaluated first (left-to-right for same level).

Note: Actually b*c and d/e are independent sub-expressions at the same precedence level; the compiler evaluates them in left-to-right order.

Q34.

What is the output of the following code?

```
printf("%d", 18/3/2);
```

(A) ✔ 3

(B) 4

(C) 6

(D) 2

Answer: (A)

Explanation: 18/3/2: left-to-right: 18/3=6, 6/2=3.

Q35.

Which of the following has the lowest precedence among arithmetic operators in C?

(A) *

(B) /

(C) ✔ +

(D) %

Answer: (C)

Explanation: + (and -) have lower precedence than *, /, and %. Among the choices, + is the lowest.

Q36.

What is the result of: 3 + 5 % 2 * 4?

(A) ✔ 7

(B) 11

(C) 3

(D) 15

Answer: (A)

Explanation: % and * are same precedence, left-to-right: 5%2=1, 1*4=4. Then 3+4=7.

Q37.

Evaluate: 20 / 4 + 3 * 2 - 8 % 3

(A) 11

(B) 12

(C) ✔ 9

(D) 10

Answer: (C)

Explanation: *, / and % are evaluated first: 20/4=5, 3*2=6, 8%3=2. Then left-to-right: 5+6=11, 11-2=9.

Q38.

What is the result of: 4 + 6 / 2 * 3 - 1?

(A) ✔ 12

(B) 11

(C) 13

(D) 14

Answer: (A)

Explanation: / and * same precedence, left-to-right: 6/2=3, 3*3=9. Then 4+9-1=12.

Q39.

Which expression uses parentheses correctly to force addition before multiplication?

(A) x * y + z

(B) ✔ x * (y + z)

(C) (x * y) + z

(D) x + y * z

Answer: (B)

Explanation: In "x * (y + z)", the parentheses force y+z to be computed before multiplication. This overrides the default precedence where * would normally execute before +.

Q40.

What is the result of: (2 + 3) * (4 - 1) / 5?

(A) ✔ 3

(B) 4

(C) 2

(D) 5

Answer: (A)

Explanation: Parentheses first: (2+3)=5, (4-1)=3. Then 5*3=15, 15/5=3.

Section 3: Associativity of Operators

Q41.
What is the associativity of arithmetic operators +, -, *, /, % in C?

(A) Right to Left
(B) ✔ Left to Right
(C) No associativity
(D) Depends on the operator

Answer: (B)

Explanation: All binary arithmetic operators (+, -, *, /, %) in C are left-to-right associative. This means when operators of the same precedence appear, evaluation proceeds from left to right.

Q42.
Due to left-to-right associativity, what is the value of 10 - 3 - 2?

(A) 9
(B) ✔ 5
(C) 7
(D) 3

Answer: (B)

Explanation: Left-to-right: (10-3)-2 = 7-2 = 5. If it were right-to-left: 10-(3-2) = 10-1 = 9 (but that is not how C works).

Q43.
What is the associativity of the assignment operator (=) in C?

(A) Left to Right
(B) ✔ Right to Left
(C) Non-associative
(D) Depends on type

Answer: (B)

Explanation: The assignment operator = in C has right-to-left associativity. This allows chained assignments like a = b = c = 5, which is evaluated as a=(b=(c=5)).

Q44.
What is the value of a after: int a, b; a = b = 10;?

(A) 0
(B) ✔ 10
(C) Undefined
(D) Garbage

Answer: (B)

Explanation: Due to right-to-left associativity of =: b=10 first, then a=10. So a=10.

Q45.

Evaluate: 16 / 4 / 2 (consider associativity)

(A) 8
(B) 4
(C) ✔ 2
(D) 1

Answer: (C)

Explanation: / is left-associative: (16/4)/2 = 4/2 = 2.

Note: If right-associative: 16/(4/2) = 16/2 = 8, but that is NOT how C works.

Q46.

In C, the expression a = b = c = 0 is valid because = is:

(A) Left-associative
(B) ✔ Right-associative
(C) Non-associative
(D) Unary

Answer: (B)

Explanation: Right-to-left associativity of = allows chaining. It is parsed as a=(b=(c=0)). c gets 0 first, then b, then a.

Q47.

What is the result of 20 - 10 - 5 in C?

(A) 15
(B) ✔ 5
(C) -5
(D) 25

Answer: (B)

Explanation: Left-to-right: (20-10)-5 = 10-5 = 5.

Q48.

What is the value of: 2 * 3 / 6 * 4?

(A) ✔ 4
(B) 1
(C) 6
(D) 2

Answer: (A)

Explanation: * and / are same precedence, left-to-right: 2*3=6, 6/6=1, 1*4=4.

Q49.

The expression x = y = 5 assigns 5 to:

(A) x only
(B) y only
(C) ✔ Both x and y
(D) Neither

Answer: (C)

Explanation: Due to right-to-left associativity: y=5 executes first (y gets 5), then x=y executes (x gets 5). Both get 5.

Q50.
What is the associativity of the unary minus (-) operator in C?

(A) Left to Right
(B) ✔ Right to Left
(C) Non-associative
(D) Bidirectional

Answer: (B)

Explanation: Unary operators in C have right-to-left associativity. For example, - - x is parsed as -(-x).

Q51.
What is the result of: 100 / 10 / 2 / 5 in C?

(A) 100
(B) 10
(C) ✔ 1
(D) 2

Answer: (C)

Explanation: Left-to-right: 100/10=10, 10/2=5, 5/5=1.

Q52.
Consider: int x = 5; x -= 3; What is x?

(A) 8
(B) -8
(C) ✔ 2
(D) -2

Answer: (C)

Explanation: x -= 3 means x = x - 3 = 5 - 3 = 2.

Q53.
What does right-to-left associativity of = mean for a = b = 5?

(A) a gets 5, then b gets a
(B) ✔ b gets 5, then a gets 5
(C) Both get 5 simultaneously
(D) Error

Answer: (B)

Explanation: Right-to-left: innermost assignment first. b=5 executes first, then a=b (which is 5). Both end up as 5.

Q54.
Which of the following correctly shows left-to-right associativity?

(A) a = b = 5
(B) -(-(x))
(C) ✔ 8 - 3 - 2 = (8-3)-2
(D) All of above

Answer: (C)

Explanation: Left-to-right associativity is shown by binary arithmetic operators. "8-3-2 = (8-3)-2" illustrates this. Assignment (=) and unary (-) are right-to-left.

Q55.
What is the result of: 8 % 5 % 3?

(A) ✔ 0
(B) 1
(C) 2
(D) 3

Answer: (A)

Explanation: % is left-associative: (8%5)%3 = 3%3 = 0.

Q56.
What is the value of: 3 * 4 / 2 * 2?

(A) ✔ 12
(B) 6
(C) 9
(D) 3

Answer: (A)

Explanation: * and / left-to-right: 3*4=12, 12/2=6, 6*2=12.

Q57.
For the expression a op a op a, if op is right-associative, how would it be parenthesised?

(A) (a op a) op a
(B) ✔ a op (a op a)
(C) Left to right
(D) Cannot determine

Answer: (B)

Explanation: Right-associativity means the rightmost operation is performed first: a op (a op a).

Q58.
In C, what is the value of: int a=10; a /= 3; after execution?

(A) 3.33
(B) ✔ 3
(C) 4
(D) 0

Answer: (B)

Explanation: a /= 3 means a = a/3 = 10/3 = 3 (integer division, truncated).

Q59.
Evaluate: 27 / 3 / 3 / 3

(A) 9
(B) ✔ 1
(C) 3
(D) 27

Answer: (B)

Explanation: Left-to-right: 27/3=9, 9/3=3, 3/3=1.

Q60.

What is the associativity of compound operators like +=, -=, *=, /=?

(A) Left to Right

(B) ✔ Right to Left

(C) Non-associative

(D) Same as their base operator

Answer: (B)

Explanation: Compound assignment operators (+=, -=, *=, /=, %=) all have right-to-left associativity, same as the simple assignment operator =.

Section 4: Expression Evaluation & Type Conversion

Q61.
What is the output of: printf("%d", (int)3.7 + (int)2.9);

(A) 6
(B) 7
(C) ✔ 5
(D) 6.6

Answer: (C)

Explanation: (int)3.7 = 3 (truncation), (int)2.9 = 2 (truncation). 3+2=5.

Q62.
What is the result of: 5 / 2 + 5.0 / 2?

(A) ✔ 4.5
(B) 5.5
(C) 5.0
(D) 4

Answer: (A)

Explanation: 5/2 = 2 (integer division, since both operands are int). 5.0/2 = 2.5 (float division). 2 + 2.5 = 4.5.

Q63.
What is the output of: printf("%d", (int)(3.5 + 2.5));

(A) ✔ 6
(B) 5
(C) 7
(D) Error

Answer: (A)

Explanation: 3.5+2.5 = 6.0. (int)6.0 = 6.

Q64.
What is implicit type conversion in C?

(A) Conversion done explicitly by programmer
(B) ✔ Automatic conversion by compiler
(C) Conversion using cast operator
(D) None of these

Answer: (B)

Explanation: Implicit type conversion (also called type coercion or promotion) is performed automatically by the C compiler when operands of different types appear in an expression. The lower type is promoted to the higher type.

Q65.

What is the output of the following code?

```
int x=5, y=2;
float r = (float)x/y;
printf("%.1f", r);
```

(A) ✔ 2.5

(B) 2

(C) 3

(D) 2.0

Answer: (A)

Explanation: x=5 (int), y=2 (int). (float)x/y: x is cast to float first, giving 5.0/2=2.5.

Q66.

What is the output of: printf("%f", 1/4 + 0.5);

(A) 0.750000

(B) ✔ 0.500000

(C) 0.250000

(D) 1.000000

Answer: (B)

Explanation: 1/4 = 0 (integer division). 0 + 0.5 = 0.5. printf prints 0.500000.

Note: Common trap: 1/4 is done in integer arithmetic before adding 0.5.

Q67.

What is arithmetic conversion/promotion in C?

(A) Converting char to string

(B) ✔ Widening smaller types for mixed expressions

(C) Narrowing float to int

(D) Signed to unsigned conversion

Answer: (B)

Explanation: When an expression mixes types (e.g., int and float), C promotes the narrower type to the wider type before performing the operation. char < int < long < float < double.

Q68.

What is the result of: (float)(5 / 2)?

(A) 2.5

(B) ✔ 2.0

(C) 3.0

(D) 2.500000

Answer: (B)

Explanation: The cast applies AFTER integer division: 5/2 = 2 first, then (float)2 = 2.0. This is different from (float)5/2 = 2.5.

Note: Placement of the cast matters greatly!

Q69.

What is the output of the following?

```
int a=5, b=2;
printf("%d", a/b);
```

(A) ✔ 2

(B) 2.500000

(C) 0

(D) 3

Answer: (A)

Explanation: a=5, b=2. a/b = 5/2 = 2 (int). printf with %d prints 2.

Q70.

In C, which type has the highest precedence in type promotion?

(A) int

(B) float

(C) ✔ double

(D) long

Answer: (C)

Explanation: In the hierarchy: char < int < long < float < double < long double. double has the highest standard promotion rank.

Q71.

What is the result of: 3.0 * (4 / 2)?

(A) ✔ 6.0

(B) 6

(C) 7.0

(D) 5.0

Answer: (A)

Explanation: Parentheses first: 4/2=2 (int). Then 3.0*2: int 2 is promoted to double, giving 6.0.

Q72.

What is the output of: printf("%d %d", 5/2, 5%2);

(A) ✔ 2 1

(B) 2.5 1

(C) 2 0

(D) 3 1

Answer: (A)

Explanation: 5/2 = 2 (integer division), 5%2 = 1. Output: 2 1.

Q73.
What happens when char is used in arithmetic in C?

(A) Error
(B) ✔ Promoted to int
(C) Stays as char
(D) Converted to float

Answer: (B)

Explanation: When a char is used in an arithmetic expression in C, it is automatically promoted to int. This is called integer promotion.

Q74.
What is the result of: (double)3 / 2?

(A) 1
(B) ✔ 1.5
(C) 1.0
(D) 2.0

Answer: (B)

Explanation: (double)3 converts 3 to 3.0 (double). Then 3.0/2: 2 is promoted to double, giving 1.5.

Q75.
What is the value of: 2 + 3.0 * 2 in C?

(A) 10.0
(B) ✔ 8.0
(C) 10
(D) 5.0

Answer: (B)

Explanation: 3.0*2 = 6.0 (2 promoted to double). Then 2+6.0: 2 promoted to double, giving 8.0.

Q76.
What is the output of: printf("%d", 5 * 4 / 2 + 3 - 1);

(A) ✔ 12
(B) 11
(C) 10
(D) 9

Answer: (A)

Explanation: 5*4=20, 20/2=10. Then 10+3-1=12.

Q77.
What is the result of: (int)7.8 % 3?

(A) ✔ 1
(B) 2
(C) 0
(D) Error

Answer: (A)

Explanation: (int)7.8 = 7. 7%3 = 1.

Q78.

Which statement about the modulus operator in C is INCORRECT?

(A) ✔ % can be used with float

(B) % gives remainder of division

(C) % has same precedence as *

(D) % is left-associative

Answer: (A)

Explanation: The modulus operator % in C can ONLY be used with integer types. Using it with float or double causes a compilation error.

Note: In C++, fmod() is used for floating-point modulus.

Q79.

What is the value of the expression: sizeof(int) * 2 + 1 (on a 32-bit system)?

(A) ✔ 9

(B) 5

(C) 7

(D) 11

Answer: (A)

Explanation: sizeof(int) = 4 on a 32-bit system. 4*2+1 = 9.

Q80.

What is the output of: printf("%.2f", 1.0/3);

(A) ✔ 0.33

(B) 0.34

(C) 0.00

(D) 1.00

Answer: (A)

Explanation: 1.0/3 = 0.333... %.2f rounds to 2 decimal places: 0.33.

Section 5: Tricky Precedence & Output Questions

Q81.

What is the output of the following C code?

```
int a = 5;
printf("%d", ++a);
```

(A) ✔ 6
(B) 5
(C) 3
(D) 2

Answer: (A)

Explanation: a=5. a++ returns 5 (post-increment), then a becomes 6. printf prints a which is now 6.

Q82.

What is the output?

```
int a = 5;
printf("%d", a++);
```

(A) ✔ 5
(B) 6
(C) 4
(D) 7

Answer: (A)

Explanation: a=5. a++ is post-increment: prints current value 5, then a becomes 6.

Q83.

What is the output of: int a=10; printf("%d", a-- - --a);

(A) 2
(B) 1
(C) 0
(D) ✔ Undefined Behavior

Answer: (D)

Explanation: Modifying a variable twice between sequence points causes undefined behavior in C. "a-- - --a" modifies a twice without a sequence point, so the result is undefined.

Note: This type of expression should be avoided in real code.

Q84.

What is the result of: 2 << 3 in C? (This is a bitwise operation for reference)

(A) 6
(B) ✔ 16
(C) 8
(D) 12

Answer: (B)

Explanation: 2 << 3 shifts 2 left by 3 bits: 2 * 2^3 = 2*8 = 16. Though not strictly arithmetic, bitwise shifts often appear in precedence questions.

Q85.

What is the output of: printf("%d", 10 / 3 * 3);

(A) 10
(B) ✔ 9
(C) 1
(D) 3

Answer: (B)

Explanation: Left-to-right: 10/3=3 (integer division). 3*3=9. This shows how integer division causes loss.

Note: Common mistake: assuming 10/3*3 = 10. Integer division truncates first!

Q86.

What is the output of: printf("%d", 5 + (int)2.9 * 2);

(A) 11
(B) ✔ 9
(C) 10
(D) 12

Answer: (B)

Explanation: (int)2.9 = 2. 2*2=4. 5+4=9.

Q87.

Evaluate: 3 + 4 * 2 / (1 - 5)

(A) 0
(B) ✔ 1
(C) -1
(D) 5

Answer: (B)

Explanation: Parentheses: (1-5) = -4. Then * and /: 4*2=8, 8/(-4)=-2. Then 3+(-2)=1.

Q88.

What is the output of: printf("%d", 4 * 5 % 3);

(A) ✔ 2
(B) 0
(C) 1
(D) 3

Answer: (A)

Explanation: * and % same precedence, left-to-right: 4*5=20, 20%3=2.

Q89.

What does the following print?

```
int x=5;
printf("%d", ++x + 0);
```

(A) 5
(B) ✔ 6
(C) 4
(D) 7

Answer: (B)

Explanation: Unary ++ has higher precedence than +. ++x: x becomes 6, returns 6. 6+0=6.

Q90.

What is the value of: 2 * (3 + 4) - 5 % 3?

(A) ✔ 12
(B) 11
(C) 14
(D) 10

Answer: (A)

Explanation: (3+4)=7. 2*7=14. 5%3=2. 14-2=12.

Q91.

What is the output of: printf("%d", 100 % 10 % 3);

(A) ✔ 0
(B) 1
(C) 3
(D) 10

Answer: (A)

Explanation: Left-to-right: 100%10=0, 0%3=0.

Q92.

What is the result of: -5 % 2 in C (C99)?

(A) 1
(B) ✔ -1
(C) 0
(D) Error

Answer: (B)

Explanation: In C99 and later, the sign of the result of % follows the dividend. -5 = 2*(-2) + (-1). So -5%2 = -1.

Note: The result of a%b has the same sign as a in C99.

Q93.
What is the output of: printf("%d", 5 + -3 * 2);

(A) ✔ -1
(B) 4
(C) -6
(D) 11

Answer: (A)

Explanation: Unary minus has higher precedence than *: -3. Then -3*2=-6. Then 5+(-6)=-1.

Q94.
What is the value of: (2 + 3) * 4 / (2 * (1 + 2))?

(A) 2
(B) ✔ 3
(C) 4
(D) 5

Answer: (B)

Explanation: (2+3)=5, (2*(1+2))=2*3=6. Then 5*4=20, 20/6=3 (integer division).

Q95.
What does this code print?

```
int a=5;
a *= 2;
printf("%d", a);
```

(A) ✔ 10
(B) 11
(C) 9
(D) 0

Answer: (A)

Explanation: a=5. a*=2 makes a=10. printf prints 10.

Q96.
What is the result of: 4 + 16 / 2 / 2 * 3 - 1?

(A) ✔ 15
(B) 16
(C) 14
(D) 17

Answer: (A)

Explanation: Left-to-right for /, *: 16/2=8, 8/2=4, 4*3=12. Then 4+12-1=15.

Q97.

Which expression is equivalent to: a * b + c * d?

(A) ✔ (a*b) + (c*d)

(B) a*(b+c)*d

(C) ((a*b)+c)*d

(D) a*((b+c)*d)

Answer: (A)

Explanation: By operator precedence, * binds tighter than +. So a*b+c*d = (a*b)+(c*d).

Q98.

What is the output of: printf("%d", 2 + 3 == 5);

(A) ✔ 1

(B) 0

(C) 5

(D) Error

Answer: (A)

Explanation: + has higher precedence than ==. 2+3=5. 5==5 is 1 (true). printf prints 1.

Note: == is a relational operator with lower precedence than +.

Q99.

What is the result of: 10 - 2 * 3 + 4 / 2?

(A) ✔ 6

(B) 7

(C) 8

(D) 5

Answer: (A)

Explanation: 2*3=6, 4/2=2. Then 10-6+2=6.

Q100.

What is the output of: printf("%d", 5 * 2 - 3 + 8 / 4 * 2);

(A) ✔ 11

(B) 12

(C) 9

(D) 13

Answer: (A)

Explanation: 5*2=10. 8/4=2, 2*2=4. Then 10-3+4=11.

Section 6: Advanced & Tricky Questions

Q101.
What is the output of: int x=7; printf("%d", x/2*2);

(A) 7
(B) ✔ 6
(C) 8
(D) 4

Answer: (B)

Explanation: Left-to-right: 7/2=3 (int division), 3*2=6.

Note: This shows x/2*2 is NOT always equal to x when x is an odd integer.

Q102.
What is the output of: printf("%d", 5==5+0);

(A) ✔ 1
(B) 0
(C) 5
(D) Error

Answer: (A)

Explanation: 5+0=5 (+ has higher precedence than ==). 5==5 is 1 (true in C).

Q103.
What is the result of: int a=3; a = a + a * a;

(A) 18
(B) ✔ 12
(C) 9
(D) 6

Answer: (B)

Explanation: RHS: a + a*a = 3 + 3*3 = 3+9 = 12. a=12.

Q104.
What is the output of: printf("%d", 3*3-2*2);

(A) ✔ 5
(B) 4
(C) 25
(D) 1

Answer: (A)

Explanation: 3*3=9, 2*2=4. 9-4=5.

Q105.

Consider: int a=2; int b = a << 1 + 1; What is b? (C precedence)

(A) ✔ 8
(B) 6
(C) 16
(D) 4

Answer: (A)

Explanation: + has higher precedence than <<. So 1+1=2 first, then a<<2 = 2<<2 = 8.

Note: Bitwise << has lower precedence than arithmetic +.

Q106.

What is the value of: (3 + 4) * (5 - 2) - (8 / 2)?

(A) ✔ 17
(B) 21
(C) 15
(D) 12

Answer: (A)

Explanation: (3+4)=7, (5-2)=3, (8/2)=4. 7*3=21, 21-4=17.

Q107.

What is the result of: int a=10; a %= 3;?

(A) 3
(B) ✔ 1
(C) 0
(D) 10

Answer: (B)

Explanation: a %= 3 means a = a%3 = 10%3 = 1.

Q108.

What is the output of: printf("%d", 5/2*2 + 1);

(A) 6
(B) ✔ 5
(C) 4
(D) 7

Answer: (B)

Explanation: 5/2=2 (integer division), 2*2=4. 4+1=5.

Note: Note: 5/2*2 ≠ 5! Integer division truncates first.

Q109.

What does this code print?

```
int a=5;
a *= a + 6;
printf("%d", a);
```

(A) ✔ 55

(B) 50

(C) 60

(D) 45

Answer: (A)

Explanation: a=5. a*=a+6: compound assignment, RHS evaluated first. a+6=11. a=5*11=55.

Q110.

Evaluate: ((5 + 3) * 2 - 4) / (6 % 4)

(A) ✔ 6

(B) 4

(C) 5

(D) 12

Answer: (A)

Explanation: (5+3)=8, 8*2=16, 16-4=12. 6%4=2. 12/2=6.

Q111.

What is the output of: printf("%d", 2*3+4*5-6/2);

(A) ✔ 23

(B) 24

(C) 25

(D) 22

Answer: (A)

Explanation: 2*3=6, 4*5=20, 6/2=3. 6+20-3=23.

Q112.

What is the result of: (float)5/2 vs 5/(float)2?

(A) ✔ 2.5 and 2.5

(B) 2 and 2.5

(C) 2.5 and 2

(D) 2 and 2

Answer: (A)

Explanation: In (float)5/2: 5 is cast to 5.0, then 5.0/2=2.5. In 5/(float)2: 2 is cast to 2.0, then 5/2.0=2.5. Both give 2.5.

Q113.
What is the output of: printf("%d", (2+3) == (4+1));

(A) 0
(B) ✔ 1
(C) 5
(D) Error

Answer: (B)

Explanation: (2+3)=5, (4+1)=5. 5==5 is true=1.

Q114.
What is the value of: 10 - -5?

(A) 5
(B) ✔ 15
(C) -15
(D) -5

Answer: (B)

Explanation: 10 - (-5) = 10 + 5 = 15. The unary minus makes -5 a negative number.

Q115.
What is the output of: int x=1; x = x + x++ * x;?

(A) 3
(B) 4
(C) 5
(D) ✔ Undefined

Answer: (D)

Explanation: This expression modifies x multiple times without sequence points, resulting in undefined behavior in C.

Q116.
Which of the following is used to override operator precedence in C?

(A) Square brackets []
(B) Curly braces {}
(C) ✔ Parentheses ()
(D) Angle brackets <>

Answer: (C)

Explanation: Parentheses () have the highest precedence in C and are used to override the default operator precedence.

Q117.
What is the result of: 7.0 / 2 + 3 / 2?

(A) ✔ 4.5
(B) 4.0
(C) 5.5
(D) 5.0

Answer: (A)

Explanation: 7.0/2 = 3.5 (floating). 3/2 = 1 (integer). 3.5+1 = 4.5.

Q118.

What is the output: printf("%d", 3 * (2 + 5) / 7);

(A) ✔ 3

(B) 2

(C) 1

(D) 6

Answer: (A)

Explanation: (2+5)=7. 3*7=21. 21/7=3.

Q119.

What is: int a=10; a = a++ + ++a;? (Assume initial a=10)

(A) 21

(B) 22

(C) 23

(D) ✔ Undefined Behavior

Answer: (D)

Explanation: This is undefined behavior. Modifying a variable multiple times without an intervening sequence point is undefined in C.

Note: Never rely on the order of ++ side effects in a single expression.

Q120.

What is the output of: printf("%d", 10 % 3 + 10 / 3);

(A) ✔ 4

(B) 3

(C) 5

(D) 6

Answer: (A)

Explanation: 10%3=1, 10/3=3. 1+3=4.

Section 7: Comprehensive Output & Evaluation

Q121.
What is the output of: printf("%d", !(5-5));

(A) 0
(B) ✔ 1
(C) 5
(D) -1

Answer: (B)

Explanation: Inside: 5-5=0. !0 = 1 (logical NOT). Output: 1.

Note: ! is logical NOT with lower prec than arithmetic.

Q122.
What is the value of: 4 * 4 / 4 + 4 * 4 % 4?

(A) 16
(B) ✔ 4
(C) 0
(D) 8

Answer: (B)

Explanation: * / % same precedence, left-to-right groups: (4*4/4)=16/4=4; (4*4%4)=16%4=0. 4+0=4.

Q123.
What is the output of: printf("%d", 5-2*3+1);

(A) ✔ 0
(B) -2
(C) 2
(D) 6

Answer: (A)

Explanation: 2*3=6. 5-6+1=0.

Q124.
Which expression always results in 0 for integer x != 0?

(A) ✔ x%x
(B) x/x
(C) x-x
(D) x*0

Answer: (A)

Explanation: x%x = 0 for any non-zero x since x divides evenly into itself. Also x-x=0 and x*0=0. But x/x=1, not 0.

Note: Answer (A) and (C) and (D) all give 0. Among options, (D) is safest.

Q125.

What is the output of: printf("%d", 2147483647 + 1); (assuming 32-bit int)?

(A) 2147483648

(B) ✔ -2147483648

(C) 0

(D) Undefined

Answer: (B)

Explanation: Adding 1 to INT_MAX (2147483647) causes signed integer overflow. In practice this wraps to -2147483648, but strictly speaking, signed overflow is undefined behavior in C.

Note: Signed integer overflow is technically undefined behavior in C!

Q126.

What is the result of: int x=7; x = x>5 ? x*2 : x+2;?

(A) ✔ 14

(B) 9

(C) 7

(D) 12

Answer: (A)

Explanation: x=7, 7>5 is true. So x = x*2 = 14.

Q127.

What does the following print?

```
int a=3, b=2;
printf("%d", a*b);
```

(A) ✔ 6

(B) 7

(C) 8

(D) 9

Answer: (A)

Explanation: a=3, b=2. a*b=6. printf prints 6.

Q128.

What is the result of: (int)(2.5 * 2)?

(A) ✔ 5

(B) 4

(C) 6

(D) Error

Answer: (A)

Explanation: 2.5*2=5.0. (int)5.0=5.

Q129.
What is the output of: printf("%d", 2+3*4-5/2+1);

(A) 12
(B) 10
(C) 11
(D) ✔ 13

Answer: (D)

Explanation: 3*4=12, 5/2=2 (integer). 2+12-2+1=13.

Q130.
How many operators are in the expression: a + b * c - d / e % f?

(A) ✔ 5
(B) 6
(C) 7
(D) 4

Answer: (A)

Explanation: Operators: +, *, -, /, % = 5 binary operators.

Q131.
What is the output of: printf("%d", 3%2 + 4/2 * 2);

(A) ✔ 5
(B) 4
(C) 3
(D) 6

Answer: (A)

Explanation: 3%2=1, 4/2=2, 2*2=4. 1+4=5.

Q132.
What is the result of: int a=5; a += a *= 2;?

(A) 20
(B) 15
(C) 25
(D) ✔ Undefined

Answer: (D)

Explanation: Multiple assignments in one expression without sequence points = undefined behavior.

Q133.
What is the precedence of () in C?

(A) ✔ Highest
(B) Lowest
(C) Same as *
(D) Same as =

Answer: (A)

Explanation: Parentheses () have the highest precedence of all operators in C, used to control evaluation order.

Q134.
What is the output of: printf("%d", 100/10/10);

(A) ✔ 1
(B) 10
(C) 100
(D) 0

Answer: (A)

Explanation: 100/10=10, 10/10=1.

Q135.
What does this code print?

```
printf("%d", (5-3)/(4-2));
```

(A) ✔ 1
(B) 2
(C) 3
(D) 4

Answer: (A)

Explanation: (5-3)=2, (4-2)=2. 2/2=1.

Q136.
What is the result of: 5 * (3 - 7) / (2 + 2)?

(A) ✔ -5
(B) 5
(C) -4
(D) -3

Answer: (A)

Explanation: (3-7)=-4, (2+2)=4. 5*(-4)=-20, -20/4=-5.

Q137.
What is the output of: printf("%d", -(-5));

(A) ✔ 5
(B) -5
(C) 0
(D) Error

Answer: (A)

Explanation: -(-5) = 5. Double negation.

Q138.
What is the value of i after: int i=5; i = i/2 + i%2;?

(A) ✔ 3
(B) 2
(C) 1
(D) 4

Answer: (A)

Explanation: i/2=2, i%2=1. 2+1=3.

Q139.

Which has higher precedence: a*b+c or a*(b+c)?

(A) ✔ a*b+c gives * precedence over +

(B) a*(b+c) forces + first

(C) (B) both same

(D) Depends on values

Answer: (A)

Explanation: By default * has higher precedence than +, so a*b+c = (a*b)+c. To make + go first, parentheses are needed: a*(b+c).

Q140.

What is the output of: printf("%d", 13/4 + 13%4);

(A) ✔ 4

(B) 5

(C) 3

(D) 6

Answer: (A)

Explanation: 13/4=3 (integer division). 13%4=1. 3+1=4.

Q141.

What is the result of: 2 * 2 + 2 * 2 == 4 * 2?

(A) ✔ 1

(B) 0

(C) 8

(D) 16

Answer: (A)

Explanation: 2*2+2*2=4+4=8. 4*2=8. 8==8 is 1.

Q142.

What is the output of: int x=5; printf("%d", x*x - x + 1);

(A) ✔ 21

(B) 20

(C) 19

(D) 16

Answer: (A)

Explanation: x*x=25, 25-5+1=21.

Q143.

What is the output of: printf("%d", 10 % 7 % 3);

(A) ✔ 0

(B) 1

(C) 3

(D) 2

Answer: (A)

Explanation: % is left-associative: 10%7=3, 3%3=0.

Q144.
What is the result of: (3+5)*(7-2)/(2+3)?

(A) ✔ 8
(B) 7
(C) 6
(D) 9

Answer: (A)

Explanation: (3+5)=8, (7-2)=5, (2+3)=5. 8*5=40, 40/5=8.

Q145.
What is the output of: printf("%d", 1+2*3-4/2+5%3);

(A) 8
(B) ✔ 7
(C) 9
(D) 6

Answer: (B)

Explanation: 2*3=6, 4/2=2, 5%3=2. 1+6-2+2=7.

Q146.
In C, what is the result type of int + float?

(A) int
(B) ✔ float
(C) double
(D) long

Answer: (B)

Explanation: When int and float are mixed, int is promoted to float. The result is float.

Q147.
What is the output of: printf("%d", 5!=5+0);

(A) ✔ 0
(B) 1
(C) 5
(D) Error

Answer: (A)

Explanation: 5+0=5 (+ higher precedence). 5!=5 is false=0.

Q148.
What is the output of: printf("%d", 12/5 + 12%5);

(A) ✔ 4
(B) 5
(C) 3
(D) 6

Answer: (A)

Explanation: 12/5=2, 12%5=2. 2+2=4.

Q149.

What is the value of: int a=4; a = ++a + a++;? (C99)

(A) 10
(B) 11
(C) 12
(D) ✔ Undefined

Answer: (D)

Explanation: Undefined behavior: a is modified twice without a sequence point.

Q150.

What is the output of: printf("%d", (5%3) * (4/2));

(A) ✔ 4
(B) 2
(C) 6
(D) 8

Answer: (A)

Explanation: 5%3=2, 4/2=2. 2*2=4.

Q151.

What happens if you apply % operator to float in C?

(A) Returns float remainder
(B) ✔ Compilation error
(C) Returns 0
(D) Undefined behavior

Answer: (B)

Explanation: The % operator is only defined for integer types in C. Using % with float operands causes a compilation error.

Q152.

What is the result of: 2^3 in C?

(A) 8
(B) ✔ 1
(C) 6
(D) Error

Answer: (B)

Explanation: ^ is the bitwise XOR operator in C, NOT exponentiation! 2 XOR 3 = 1 (010 XOR 011 = 001).

Note: In C, there is no ** or ^ for power. Use pow() from math.h.

Q153.

What is the result of: printf("%d", 50/8*8+50%8);

(A) ✔ 50

(B) 48

(C) 52

(D) 54

Answer: (A)

Explanation: 50/8=6, 6*8=48, 50%8=2. 48+2=50.

Note: This shows that (x/y)*y + x%y == x (for positive integers).

Q154.

What is the output of: int a=2,b=3; printf("%d", a*b+b*a);

(A) ✔ 12

(B) 10

(C) 6

(D) 13

Answer: (A)

Explanation: a*b=6, b*a=6. 6+6=12.

Q155.

What is a "side effect" in C arithmetic?

(A) Output to screen

(B) ✔ Modification of a variable by ++ or --

(C) Division by zero

(D) Type conversion

Answer: (B)

Explanation: Side effects are changes to program state (like variable modification) caused by operators. Operators like ++ and -- have side effects as they modify variables.

Q156.

What is the output of: printf("%d", (int)(9.99));

(A) 10

(B) ✔ 9

(C) 0

(D) Error

Answer: (B)

Explanation: (int) truncates: (int)9.99 = 9.

Q157.

What does the C standard say about signed integer overflow?

(A) Wraps around

(B) ✔ Undefined behavior

(C) Saturates to MAX

(D) Zero

Answer: (B)

Explanation: Signed integer overflow is undefined behavior in C. The compiler may produce any result.

Q158.
What is the result of: 3 * 4 % 5?

(A) ✔ 2
(B) 3
(C) 7
(D) 1

Answer: (A)

Explanation: * and % same precedence, left-to-right: 3*4=12, 12%5=2.

Q159.
What is the output of: printf("%d", (3+4==7)*5);

(A) ✔ 5
(B) 1
(C) 7
(D) 35

Answer: (A)

Explanation: 3+4=7. 7==7 is 1. 1*5=5.

Q160.
In C, what is the result of: printf("%d", ~0);?

(A) ✔ -1
(B) 0
(C) 1
(D) Undefined

Answer: (A)

Explanation: ~0 is bitwise NOT of 0. All bits flip from 0 to 1, giving -1 in two's complement.

Q161.
What is the value of: ((int)(2.9 + 2.9))?

(A) ✔ 5
(B) 6
(C) 4
(D) Error

Answer: (A)

Explanation: 2.9+2.9=5.8. (int)5.8=5.

Q162.
What is the output of: int x=3; printf("%d", x*x+2*x+1);

(A) ✔ 16
(B) 15
(C) 18
(D) 20

Answer: (A)

Explanation: x*x=9, 2*x=6. 9+6+1=16.

Q163.
Which operator has the same precedence as *?

(A) + (binary)
(B) ✔ % (modulus)
(C) = (assignment)
(D) - (unary)

Answer: (B)

Explanation: * and % (and /) are all at the same precedence level in C.

Q164.
What is the output of: printf("%d", 20/(2+3)*5);

(A) ✔ 20
(B) 10
(C) 25
(D) 50

Answer: (A)

Explanation: (2+3)=5. 20/5=4. 4*5=20.

Q165.
What is the result of: int x=10; x = x + (x=5);?

(A) 15
(B) 10
(C) 20
(D) ✔ Undefined

Answer: (D)

Explanation: Undefined behavior: x is read and modified in the same expression without a sequence point.

Q166.
What is the output of: printf("%d", 5/2 == 2);

(A) ✔ 1
(B) 0
(C) 2
(D) 5

Answer: (A)

Explanation: 5/2=2 (int). 2==2 is 1 (true).

Q167.
The expression a*b/c*d when computed left-to-right could differ from a*b*d/c when:

(A) Never
(B) ✔ When c does not divide b
(C) Always
(D) Only for floats

Answer: (B)

Explanation: For integers: a*b/c*d vs a*b*d/c can differ if integer division causes truncation at different points.

Q168.

What is the output of: printf("%f", (float)(3/2));

(A) ✔ 1.000000

(B) 1.500000

(C) 0.000000

(D) Error

Answer: (A)

Explanation: 3/2=1 (integer division first). (float)1=1.0. Output: 1.000000.

Note: The cast comes after division! Use (float)3/2 for 1.5.

Q169.

What is the result of: 5*2%3+1?

(A) 3

(B) ✔ 2

(C) 5

(D) 4

Answer: (B)

Explanation: * and % same precedence, left-to-right: 5*2=10, 10%3=1. Then 1+1=2.

Q170.

What is the output of: int x=100; x /= 3; x *= 3; printf("%d",x);

(A) 100

(B) ✔ 99

(C) 101

(D) 102

Answer: (B)

Explanation: x/=3: 100/3=33 (int). x*=3: 33*3=99. Prints 99.

Note: Integer division causes loss; x/=3*=3 does not recover original x.

Q171.

What is the output of: printf("%d", (2+3)*4-(6/2)*2+1);

(A) 14

(B) ✔ 15

(C) 16

(D) 13

Answer: (B)

Explanation: (2+3)=5, 5*4=20. (6/2)=3, 3*2=6. 20-6+1=15.

Q172.

What is the output: printf("%d", 0x10 / 4);

(A) 1

(B) ✔ 4

(C) 16

(D) Error

Answer: (B)

Explanation: 0x10 is hexadecimal 16. 16/4=4.

Q173.

What is the output: int a=5; printf("%d", a*=2, a);

(A) 10 10
(B) 5 10
(C) 10 5
(D) ✔ Undefined

Answer: (D)

Explanation: Order of evaluation of function arguments is unspecified in C, leading to undefined behavior.

Q174.

What is the correct formula to compute hypotenuse in C for int sides?

(A) h = a*a + b*b
(B) ✔ h = sqrt(a*a + b*b)
(C) h = pow(a+b, 2)
(D) h = (a+b)*(a+b)

Answer: (B)

Explanation: sqrt(a*a + b*b) correctly computes the hypotenuse using the Pythagorean theorem. Include math.h and link with -lm.

Q175.

What is the output of: printf("%d", (3>2) + (4>3) + (5>4));

(A) ✔ 3
(B) 2
(C) 1
(D) 0

Answer: (A)

Explanation: 3>2=1, 4>3=1, 5>4=1. 1+1+1=3.

Mock Test 1

Full-Length Practice — 25 Questions | Time: 30 Minutes

Instructions:

- Each question carries 1 mark.
- No negative marking unless specified by exam body.
- Choose the best option among (A), (B), (C), (D).
- Cover the answer column and attempt independently before checking.

Q1.
What is 7/2 in C?

(A) 3.5
(B) ✔ 3
(C) 4
(D) 2

Answer: (B)

Explanation: 7 and 2 are int; integer division gives 3.

Q2.
Evaluate: 3+4*2

(A) 14
(B) 10
(C) ✔ 11
(D) 24

Answer: (C)

Explanation: 4*2=8 (higher prec). 3+8=11.

Q3.
What is 10-2*3?

(A) 24
(B) ✔ 4
(C) 8
(D) 6

Answer: (B)

Explanation: 2*3=6, 10-6=4.

Q4.
Result of 15%4?

(A) 4
(B) ✔ 3
(C) 2
(D) 1

Answer: (B)

Explanation: 15=4*3+3, so 15%4=3.

Q5.
What is (int)(7.9)?

(A) 8
(B) ✔ 7
(C) 7.0
(D) Error

Answer: (B)

Explanation: (int) truncates: 7.

Q6.
Which is left-associative?

(A) ✔ +,-
(B) =
(C) Unary -
(D) All

Answer: (A)

Explanation: Binary + and - are left-associative.

Q7.
Value of: 16/4/2?

(A) 8
(B) 4
(C) ✔ 2
(D) 1

Answer: (C)

Explanation: 16/4=4, 4/2=2.

Q8.
x=5; x+=3; x=?

(A) ✔ 8
(B) 15
(C) 2
(D) 5

Answer: (A)

Explanation: x=5+3=8.

Q9.
Which is NOT arithmetic?

(A) +
(B) *
(C) ✔ &&
(D) %

Answer: (C)

Explanation: && is logical AND.

Q10.
Result of (float)5/2?

(A) 2
(B) ✔ 2.5
(C) 2.0
(D) Error

Answer: (B)

Explanation: (float)5=5.0, 5.0/2=2.5.

Q11.
What is -5%2 in C99?

(A) 1
(B) ✔ -1
(C) 0
(D) Error

Answer: (B)

Explanation: Sign follows dividend: -1.

Q12.
Value of 2+3*4-5/2?

(A) 11
(B) ✔ 12
(C) 13
(D) 10

Answer: (B)

Explanation: 3*4=12, 5/2=2. 2+12-2=12.

Q13.
Result of 100/10/10?

(A) ✔ 1
(B) 10
(C) 100
(D) 0

Answer: (A)

Explanation: 100/10=10, 10/10=1.

Q14.
Associativity of = is?

(A) L-to-R
(B) ✔ R-to-L
(C) None
(D) Bidirectional

Answer: (B)

Explanation: Assignment is right-to-left.

Q15.
Value of: 5*2+3*4?

(A) 46
(B) ✔ 22
(C) 34
(D) 26

Answer: (B)

Explanation: 5*2=10, 3*4=12. 10+12=22.

Q16.
What is (int)(3.5+2.5)?

(A) ✔ 6
(B) 5
(C) 7
(D) Error

Answer: (A)

Explanation: 3.5+2.5=6.0. (int)6.0=6.

Q17.
Result of 3*4%5?

(A) ✔ 2
(B) 7
(C) 3
(D) 1

Answer: (A)

Explanation: 3*4=12, 12%5=2.

Q18.
5/2==2 in C is?

(A) ✔ 1
(B) 0
(C) 5
(D) 2

Answer: (A)

Explanation: 5/2=2 (int). 2==2=1 (true).

Q19.
Which overrides precedence?

(A) []
(B) {}
(C) ✔ ()
(D) <>

Answer: (C)

Explanation: Parentheses.

Q20.
Result of 3+5%2*4?

(A) ✔ 7
(B) 11
(C) 3
(D) 15

Answer: (A)

Explanation: 5%2=1, 1*4=4. 3+4=7.

Q21.
Value of: 2*3/6*4?

(A) ✔ 4
(B) 1
(C) 6
(D) 2

Answer: (A)

Explanation: 2*3=6, 6/6=1, 1*4=4.

Q22.
Output: printf("%d",50/8*8+50%8)?

(A) ✔ 50
(B) 48
(C) 52
(D) 54

Answer: (A)

Explanation: (50/8)*8+50%8=48+2=50.

Q23.
What is int+float result type?

(A) int
(B) ✔ float
(C) double
(D) long

Answer: (B)

Explanation: Promoted to float.

Q24.

Unary - associativity?

(A) L-to-R

(B) ✔ R-to-L

(C) None

(D) Both

Answer: (B)

Explanation: Unary ops are right-to-left.

Q25.

Result: (3+5)*(7-2)/(2+3)?

(A) ✔ 8

(B) 6

(C) 10

(D) 4

Answer: (A)

Explanation: 8*5/5=8.

Mock Test 2

Full-Length Practice — 25 Questions | Time: 30 Minutes

Instructions:

• Each question carries 1 mark. Time yourself strictly.

• Attempt all questions. Evaluate your score against the key.

Q1.

What does 7.0/2 evaluate to?

(A) 3

(B) ✔ 3.5

(C) 4

(D) 3.0

Answer: (B)

Explanation: 7.0 is float, 7.0/2=3.5.

Q2.

Evaluate: 20/4+3*2-8%3

(A) ✔ 9

(B) 11

(C) 7

(D) 12

Answer: (A)

Explanation: 20/4=5, 3*2=6, 8%3=2. 5+6-2=9.

Q3.

What is 13%5?

(A) 2

(B) ✔ 3

(C) 4

(D) 1

Answer: (B)

Explanation: 13=5*2+3, so 13%5=3.

Q4.
Value of: 1/4*4?

(A) 1
(B) 4
(C) ✔ 0
(D) 16

Answer: (C)

Explanation: 1/4=0 (int), 0*4=0.

Q5.
Result of (float)(3/2)?

(A) 1.5
(B) ✔ 1.0
(C) 0.5
(D) Error

Answer: (B)

Explanation: 3/2=1 first, then (float)1=1.0.

Q6.
Which converts int to float in C?

(A) ✔ (float)x
(B) float(x)
(C) toFloat(x)
(D) f(x)

Answer: (A)

Explanation: C uses (type)expr cast syntax.

Q7.
Value of: 10-3-2?

(A) 9
(B) ✔ 5
(C) 3
(D) 7

Answer: (B)

Explanation: Left-to-right: (10-3)-2=5.

Q8.
Output: printf("%d",3+4-2+1)?

(A) ✔ 6
(B) 7
(C) 4
(D) 5

Answer: (A)

Explanation: 3+4=7,7-2=5,5+1=6.

Q9.
% operator works with?

(A) float
(B) ✔ int only
(C) double
(D) char

Answer: (B)

Explanation: % is for integers only.

Q10.
int a=3; a=a+a*a; Value of a?

(A) 18
(B) ✔ 12
(C) 9
(D) 6

Answer: (B)

Explanation: a+a*a = 3+3*3 = 3+9 = 12.

Q11.
What is sizeof(int)*2+1 on 32-bit?

(A) ✔ 9
(B) 5
(C) 7
(D) 11

Answer: (A)

Explanation: sizeof(int)=4. 4*2+1=9.

Q12.
Output: printf("%d", !(5-5))?

(A) 0
(B) ✔ 1
(C) 5
(D) -1

Answer: (B)

Explanation: 5-5=0, !0=1.

Q13.
Value of: 27/3/3/3?

(A) 9
(B) ✔ 1
(C) 3
(D) 27

Answer: (B)

Explanation: 27/3=9,9/3=3,3/3=1.

Q14.
What is the precedence of unary minus vs *?

(A) Same
(B) ✔ Unary higher
(C) * higher
(D) Depends

Answer: (B)

Explanation: Unary minus has higher precedence than *.

Q15.
Result of -3*4+2?

(A) -14
(B) ✔ -10
(C) 14
(D) -8

Answer: (B)

Explanation: (-3)*4=-12, -12+2=-10.

Q16.
Value: 4*4/4+4*4%4?

(A) ✔ 4
(B) 16
(C) 8
(D) 0

Answer: (A)

Explanation: 16/4=4, 16%4=0. 4+0=4.

Q17.
What is 8%5%3?

(A) ✔ 0
(B) 1
(C) 2
(D) 3

Answer: (A)

Explanation: 8%5=3, 3%3=0.

Q18.
Output: printf("%d", (2+3)==(4+1))?

(A) ✔ 1
(B) 0
(C) 5
(D) Error

Answer: (A)

Explanation: 5==5=1.

Q19.
a=b=5 uses which associativity?

(A) L-to-R
(B) ✔ R-to-L
(C) None
(D) Mixed

Answer: (B)

Explanation: = is right-to-left.

Q20.
Value: 3+4*2/(1-5)?

(A) ✔ 1
(B) 0
(C) -1
(D) 5

Answer: (A)

Explanation: (1-5)=-4, 4*2=8, 8/(-4)=-2. 3+(-2)=1.

Q21.
Output: printf("%d",10/3*3)?

(A) 10
(B) ✔ 9
(C) 1
(D) 3

Answer: (B)

Explanation: 10/3=3, 3*3=9.

Q22.
Result of (3>2)+(4>3)+(5>4)?

(A) ✔ 3
(B) 2
(C) 1
(D) 0

Answer: (A)

Explanation: 1+1+1=3.

Q23.
^ in C is?

(A) Power
(B) ✔ XOR
(C) NOT
(D) OR

Answer: (B)

Explanation: ^ is bitwise XOR. Use pow() for exponentiation.

Q24.

Result: printf("%d", (int)(2.5*2))?

(A) ✔ 5

(B) 4

(C) 6

(D) Error

Answer: (A)

Explanation: 2.5*2=5.0, (int)5.0=5.

Q25.

Value: 0x10/4?

(A) 1

(B) ✔ 4

(C) 16

(D) Error

Answer: (B)

Explanation: 0x10=16, 16/4=4.

Answer Key — Main Questions (Q1–Q175)

Q1: (A)	**Q2**: (B)	**Q3**: (B)	**Q4**: (B)	**Q5**: (C)
Q6: (A)	**Q7**: (B)	**Q8**: (B)	**Q9**: (C)	**Q10**: (A)
Q11: (B)	**Q12**: (A)	**Q13**: (B)	**Q14**: (B)	**Q15**: (C)
Q16: (B)	**Q17**: (B)	**Q18**: (C)	**Q19**: (A)	**Q20**: (B)
Q21: (B)	**Q22**: (B)	**Q23**: (A)	**Q24**: (B)	**Q25**: (A)
Q26: (D)	**Q27**: (A)	**Q28**: (A)	**Q29**: (A)	**Q30**: (C)
Q31: (B)	**Q32**: (A)	**Q33**: (B)	**Q34**: (A)	**Q35**: (C)
Q36: (A)	**Q37**: (C)	**Q38**: (A)	**Q39**: (B)	**Q40**: (A)
Q41: (B)	**Q42**: (B)	**Q43**: (B)	**Q44**: (B)	**Q45**: (C)
Q46: (B)	**Q47**: (B)	**Q48**: (A)	**Q49**: (C)	**Q50**: (B)
Q51: (C)	**Q52**: (C)	**Q53**: (B)	**Q54**: (C)	**Q55**: (A)
Q56: (A)	**Q57**: (B)	**Q58**: (B)	**Q59**: (B)	**Q60**: (B)
Q61: (C)	**Q62**: (A)	**Q63**: (A)	**Q64**: (B)	**Q65**: (A)
Q66: (B)	**Q67**: (B)	**Q68**: (B)	**Q69**: (A)	**Q70**: (C)
Q71: (A)	**Q72**: (A)	**Q73**: (B)	**Q74**: (B)	**Q75**: (B)
Q76: (A)	**Q77**: (A)	**Q78**: (A)	**Q79**: (A)	**Q80**: (A)
Q81: (A)	**Q82**: (A)	**Q83**: (D)	**Q84**: (B)	**Q85**: (B)
Q86: (B)	**Q87**: (B)	**Q88**: (A)	**Q89**: (B)	**Q90**: (A)
Q91: (A)	**Q92**: (B)	**Q93**: (A)	**Q94**: (B)	**Q95**: (A)
Q96: (A)	**Q97**: (A)	**Q98**: (A)	**Q99**: (A)	**Q100**: (A)
Q101: (B)	**Q102**: (A)	**Q103**: (B)	**Q104**: (A)	**Q105**: (A)
Q106: (A)	**Q107**: (B)	**Q108**: (B)	**Q109**: (A)	**Q110**: (A)
Q111: (A)	**Q112**: (A)	**Q113**: (B)	**Q114**: (B)	**Q115**: (D)
Q116: (C)	**Q117**: (A)	**Q118**: (A)	**Q119**: (D)	**Q120**: (A)
Q121: (B)	**Q122**: (B)	**Q123**: (A)	**Q124**: (A)	**Q125**: (B)
Q126: (A)	**Q127**: (A)	**Q128**: (A)	**Q129**: (D)	**Q130**: (A)
Q131: (A)	**Q132**: (D)	**Q133**: (A)	**Q134**: (A)	**Q135**: (A)
Q136: (A)	**Q137**: (A)	**Q138**: (A)	**Q139**: (A)	**Q140**: (A)
Q141: (A)	**Q142**: (A)	**Q143**: (A)	**Q144**: (A)	**Q145**: (B)
Q146: (B)	**Q147**: (A)	**Q148**: (A)	**Q149**: (D)	**Q150**: (A)
Q151: (B)	**Q152**: (B)	**Q153**: (A)	**Q154**: (A)	**Q155**: (B)
Q156: (B)	**Q157**: (B)	**Q158**: (A)	**Q159**: (A)	**Q160**: (A)

Q161: (A)	**Q162**: (A)	**Q163**: (B)	**Q164**: (A)	**Q165**: (D)
Q166: (A)	**Q167**: (B)	**Q168**: (A)	**Q169**: (B)	**Q170**: (B)
Q171: (B)	**Q172**: (B)	**Q173**: (D)	**Q174**: (B)	**Q175**: (A)

Mock Test 1 — Answer Key

Q1: (B)	**Q2**: (C)	**Q3**: (B)	**Q4**: (B)	**Q5**: (B)
Q6: (A)	**Q7**: (C)	**Q8**: (A)	**Q9**: (C)	**Q10**: (B)
Q11: (B)	**Q12**: (B)	**Q13**: (A)	**Q14**: (B)	**Q15**: (B)
Q16: (A)	**Q17**: (A)	**Q18**: (A)	**Q19**: (C)	**Q20**: (A)
Q21: (A)	**Q22**: (A)	**Q23**: (B)	**Q24**: (B)	**Q25**: (A)

Mock Test 2 — Answer Key

Q1: (B)	**Q2**: (A)	**Q3**: (B)	**Q4**: (C)	**Q5**: (B)
Q6: (A)	**Q7**: (B)	**Q8**: (A)	**Q9**: (B)	**Q10**: (B)
Q11: (A)	**Q12**: (B)	**Q13**: (B)	**Q14**: (B)	**Q15**: (B)
Q16: (A)	**Q17**: (A)	**Q18**: (A)	**Q19**: (B)	**Q20**: (A)
Q21: (B)	**Q22**: (A)	**Q23**: (B)	**Q24**: (A)	**Q25**: (B)

Key Concept Notes & Exam Tips

★ Integer Division Trap

In C, 5/2 = 2, NOT 2.5. When both operands are int, the result is int with truncation. To get float result, cast at least one operand: (float)5/2 = 2.5.

★ Modulus Operator (%)

Only works with integers in C. Result sign follows the dividend (C99): -7%2 = -1. For x%y, result is in range [-(|y|-1), |y|-1].

★ Operator Precedence Mnemonic

Remember: "Parentheses Unary Mult-Div-Mod Add-Sub Assignment" = Parentheses > Unary > (* / %) > (+ -) > Assignment.

★ Left-to-Right vs Right-to-Left

Binary arithmetic (+,-,*,/,%) are LEFT-associative: a-b-c = (a-b)-c. Assignment (=, +=, etc.) is RIGHT-associative: a=b=5 means a=(b=5).

★ Undefined Behavior (UB)

Expressions like i++ + ++i or a = a++ modify a variable twice without a sequence point. This is UNDEFINED BEHAVIOR in C — avoid such expressions in exams and real code!

★ Type Promotion Rules

When types mix: char → int → long → float → double. The lower type is always promoted. Example: int+float = float, int+double = double.

★ Cast Operator Placement

(float)(3/2) = (float)1 = 1.0 (division happens first!). vs (float)3/2 = 3.0/2 = 1.5 (cast applied before division). Placement matters!

★ ^ is NOT Power in C

In C, ^ is the bitwise XOR operator, NOT exponentiation! 2^3 = 1 (bitwise), not 8. Use pow(2,3) from math.h for exponentiation.

Score Interpretation — Mock Tests (25 Marks)

Score Range	Performance	Recommendation
22 – 25	Excellent ★★★	You are well-prepared for this topic!

18 – 21	Good ★★	Revisit tricky precedence and UB questions.
13 – 17	Average ★	Re-read concept notes; practice more output Qs.
Below 13	Needs Improvement	Go through all sections systematically again.

Best Wishes for Your OAVS TGT Exam!

This book covers all key aspects of Arithmetic Expressions in C as required for the TGT Computer Science examination.

Topics covered: Basic expressions · Operator Precedence (5 levels) · Left/Right Associativity · Integer vs Float arithmetic · Type promotion · Cast operator · Undefined behavior · Tricky output questions · 2 full mock tests.

Prepared as Prompt 7 of the OAVS TGT Computer Science Exam Preparation Series.